

Министерство науки и высшего образования Российской Федерации

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ

ИТМО

ITMO University

ЛАБОРАТОРНАЯ РАБОТА №3

По дисциплине Алгоритмы и структуры данных

Тема работы Алгоритмы сортировки

Обучающийся Черненко Андрей Дмитриевич

Факультет Факультет прикладной информатики

Группа K4111c

Образовательная программа Разработка и эксплуатация промышленного
бэкенда

Обучающийся

(дата)

(подпись)

Черненко А. Д.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Мусаев А. А.

(Ф.И.О.)

Санкт–Петербург, 2026

Содержание

ВВЕДЕНИЕ	2
1 Задание 1. Быстрая сортировка, сортировка слиянием и расчёской	2
1.1 Условие	2
1.2 Реализация	3
1.2.1 Быстрая сортировка	3
1.2.2 Сортировка слиянием	3
1.2.3 Сортировка расчёской	4
1.3 Результат работы	5
2 Задание 2. Блочная и пирамидальная сортировки	5
2.1 Условие	5
2.2 Блочная сортировка	5
2.3 Пирамидальная сортировка	6
2.4 Результат работы	7
3 Сравнение алгоритмов	8
ЗАКЛЮЧЕНИЕ	8

ВВЕДЕНИЕ

Целью лабораторной работы является изучение алгоритмов сортировки, принципов их работы и асимптотической сложности. Сортировка является одной из наиболее распространённых операций обработки данных, поэтому выбор подходящего алгоритма существенно влияет на производительность программы.

В ходе работы рассматриваются пять алгоритмов: быстрая сортировка, сортировка слиянием, сортировка расчёской, блочная сортировка и пирамидальная сортировка. Для каждого алгоритма реализована отдельная функция на языке C#.

В работе решаются следующие задачи:

1. реализовать функции быстрой сортировки, сортировки слиянием и сортировки расчёской;
2. изучить и реализовать блочную и пирамидальную сортировки;
3. сравнить алгоритмы по времени работы, дополнительной памяти, достоинствам и недостаткам.

Все программы реализованы на языке C# с использованием .NET 8. Каждый пункт задания оформлен отдельным проектом в каталоге Labs/Lab03.

1 Задание 1. Быстрая сортировка, сортировка слиянием и расчёской

1.1 Условие

Необходимо написать программы с функциями для быстрой сортировки, сортировки слиянием и сортировки расчёской. Алгоритмы должны упорядочивать элементы массива по возрастанию.

1.2 Реализация

1.2.1 Быстрая сортировка

Быстрая сортировка выбирает опорный элемент и разделяет массив на две части: элементы, меньшие опорного, и элементы, большие опорного. Затем обе части сортируются рекурсивно.

```
1 private static void QuickSort(int[] array, int left, int right)
2 {
3     if (left >= right) return;
4
5     var pivot = array[left + (right - left) / 2];
6     var i = left;
7     var j = right;
8
9     while (i <= j)
10    {
11        while (array[i] < pivot) i++;
12        while (array[j] > pivot) j--;
13
14        if (i <= j)
15        {
16            (array[i], array[j]) = (array[j], array[i]);
17            i++;
18            j--;
19        }
20    }
21
22    if (left < j) QuickSort(array, left, j);
23    if (i < right) QuickSort(array, i, right);
24 }
```

Листинг 1: Функция быстрой сортировки

Средняя сложность алгоритма составляет $O(n \log n)$, поскольку на каждом уровне рекурсии массив делится примерно пополам. В худшем случае, когда разбиение сильно несбалансировано, сложность равна $O(n^2)$. Дополнительная память в среднем составляет $O(\log n)$ из-за стека рекурсии.

1.2.2 Сортировка слиянием

Сортировка слиянием рекурсивно делит массив на две половины, сортирует каждую половину и объединяет отсортированные части.

```
1 private static void MergeSort(int[] array, int[] buffer, int left, int right)
```

```

2 {
3     if (left >= right) return;
4
5     var middle = left + (right - left) / 2;
6     MergeSort(array, buffer, left, middle);
7     MergeSort(array, buffer, middle + 1, right);
8
9     var i = left;
10    var j = middle + 1;
11    var k = left;
12
13    while (i <= middle && j <= right)
14        buffer[k++] = array[i] <= array[j] ? array[i++] : array[j++];
15    while (i <= middle) buffer[k++] = array[i++];
16    while (j <= right) buffer[k++] = array[j++];
17
18    for (var index = left; index <= right; index++)
19        array[index] = buffer[index];
20 }

```

Листинг 2: Функция сортировки слиянием

На каждом уровне рекурсии выполняется обработка всех n элементов, а количество уровней равно $\log n$. Поэтому сложность алгоритма в лучшем, среднем и худшем случаях равна $O(n \log n)$. Для временного массива требуется $O(n)$ дополнительной памяти.

1.2.3 Сортировка расчёской

Сортировка расчёской является развитием пузырьковой сортировки. В начале сравниваются элементы, находящиеся на большом расстоянии друг от друга. Затем расстояние уменьшается примерно в 1,3 раза, пока не станет равным единице.

```

1 private static void CombSort(int[] array)
2 {
3     var gap = array.Length;
4     var swapped = true;
5
6     while (gap > 1 || swapped)
7     {
8         gap = Math.Max(1, (int)(gap / 1.3));
9         swapped = false;
10
11         for (var i = 0; i + gap < array.Length; i++)
12         {

```

```

13         if (array[i] <= array[i + gap]) continue;
14         (array[i], array[i + gap]) =
15             (array[i + gap], array[i]);
16         swapped = true;
17     }
18 }
19 }

```

Листинг 3: Функция сортировки расчёской

Алгоритм работает непосредственно в исходном массиве и использует $O(1)$ дополнительной памяти. Для типичных наборов данных его средняя сложность оценивается как $O(n^2)$, а в удачных случаях может приближаться к $O(n \log n)$. В отличие от сортировки слиянием, строгой гарантии $O(n \log n)$ нет.

1.3 Результат работы

Для проверки использовался массив:

$$\{42, -7, 15, 0, 23, 15, 8, -20, 4\}.$$

Все три алгоритма сформировали одинаковый отсортированный массив:

$$\{-20, -7, 0, 4, 8, 15, 15, 23, 42\}.$$

2 Задание 2. Блочная и пирамидальная сортировки

2.1 Условие

Необходимо изучить блочную и пирамидальную сортировки и реализовать соответствующие программы.

2.2 Блочная сортировка

Блочная сортировка распределяет элементы по нескольким блокам в зависимости от их значения. После распределения каждый блок сортируется отдельно, а затем блоки последовательно объединяются в исходный массив.

```

1 private static void BucketSort(int[] array)
2 {
3     if (array.Length < 2) return;
4
5     var minimum = array.Min();
6     var maximum = array.Max();
7     if (minimum == maximum) return;
8
9     var buckets = new List<int>[Math.Max(1,
10         (int)Math.Sqrt(array.Length))];
11     for (var i = 0; i < buckets.Length; i++) buckets[i] = [];
12
13     var range = (long)maximum - minimum + 1;
14     foreach (var value in array)
15     {
16         var index = (int)((long)value - minimum)
17             * buckets.Length / range;
18         buckets[Math.Min(index, buckets.Length - 1)]
19             .Add(value);
20     }
21
22     var position = 0;
23     foreach (var bucket in buckets)
24     {
25         bucket.Sort();
26         foreach (var value in bucket)
27             array[position++] = value;
28     }
29 }

```

Листинг 4: Функция блочной сортировки

При равномерном распределении элементов по блокам средняя сложность составляет $O(n + k)$, где k — количество блоков. Если все элементы попадут в один блок, сортировка блока может занять $O(n^2)$ в зависимости от используемого метода. Дополнительная память составляет $O(n + k)$.

2.3 Пирамидальная сортировка

Пирамидальная сортировка сначала строит двоичную кучу, в которой каждый родитель не меньше своих потомков. Затем максимальный элемент перемещается в конец массива, после чего куча восстанавливается.

```

1 private static void HeapSort(int[] array)
2 {

```

```

3     for (var i = array.Length / 2 - 1; i >= 0; i--)
4         SiftDown(array, i, array.Length);
5
6     for (var end = array.Length - 1; end > 0; end--)
7     {
8         (array[0], array[end]) = (array[end], array[0]);
9         SiftDown(array, 0, end);
10    }
11 }
12
13 private static void SiftDown(int[] array, int root, int length)
14 {
15     while (true)
16     {
17         var child = root * 2 + 1;
18         if (child >= length) return;
19         if (child + 1 < length &&
20             array[child] < array[child + 1])
21             child++;
22         if (array[root] >= array[child]) return;
23
24         (array[root], array[child]) =
25             (array[child], array[root]);
26         root = child;
27     }
28 }

```

Листинг 5: Функции пирамидальной сортировки

Построение кучи выполняется за $O(n)$, а каждое из n извлечений требует $O(\log n)$. Поэтому лучшая, средняя и худшая сложность алгоритма равны $O(n \log n)$. Сортировка выполняется на месте и использует $O(1)$ дополнительной памяти.

2.4 Результат работы

Для того же тестового массива оба алгоритма получили результат:

$$\{-20, -7, 0, 4, 8, 15, 15, 23, 42\}.$$

3 Сравнение алгоритмов

Таблица 1: Сравнение алгоритмов сортировки

Алгоритм	Лучший	Средний	Худший	Память
Быстрая сортировка	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Сортировка слиянием	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Сортировка расчёской	$O(n \log n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Блочная сортировка	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$
Пирамидальная сортировка	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были изучены и реализованы пять алгоритмов сортировки. Все алгоритмы успешно обработали тестовый массив, содержащий отрицательные значения и повторяющиеся элементы.

Быстрая сортировка показывает высокую среднюю производительность, но в худшем случае может работать за $O(n^2)$. Сортировка слиянием имеет гарантированную сложность $O(n \log n)$, однако требует дополнительной памяти. Сортировка расчёской проста и работает на месте, но не имеет такой же гарантии.

Блочная сортировка особенно эффективна при равномерном распределении значений, тогда как её результат зависит от диапазона и распределения данных. Пирамидальная сортировка обеспечивает гарантированную сложность $O(n \log n)$ и использует $O(1)$ дополнительной памяти, поэтому является универсальным вариантом для случаев, когда важны предсказуемость и экономия памяти.